

Un sistema de gestión para una tienda de cartas Magic The Gathering

Asignatura: Desarrollo Fullstack I

Integrantes: Ignacia Padilla y Elba Sánchez

Docente: Ricardo Mauricio Gonzalez Vejar

1. Introducción y Contexto.

1.1 Descripción del proyecto.

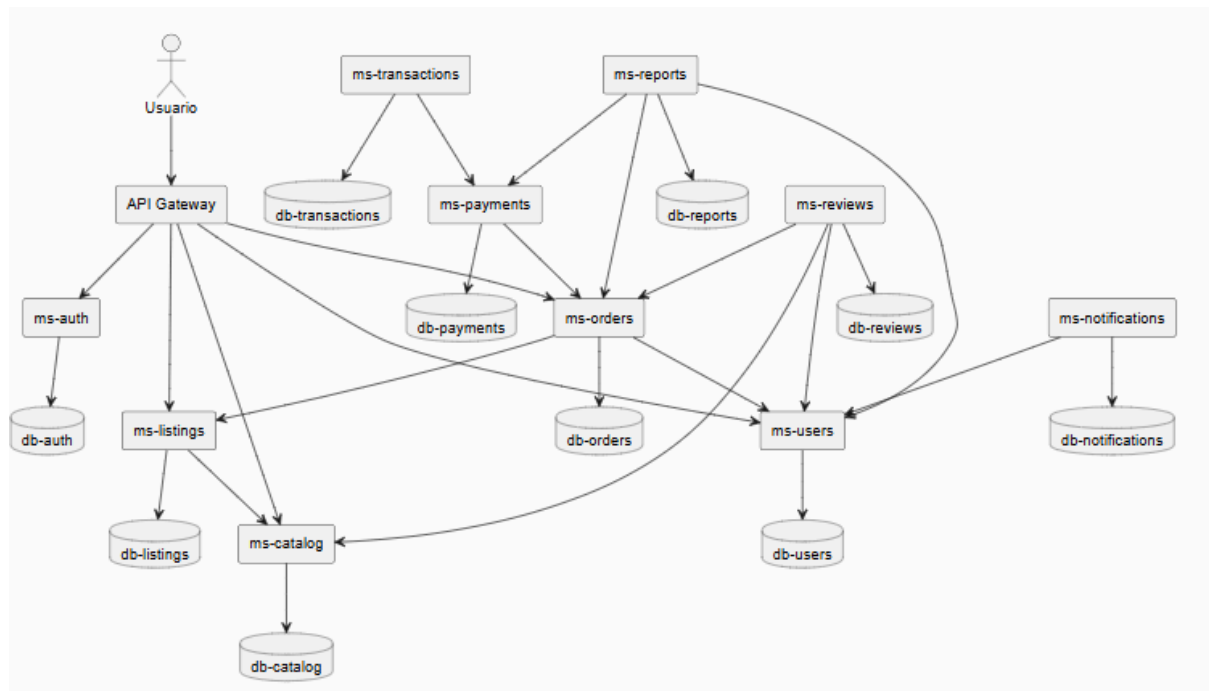
Quandrix es una solución de backend basada en una arquitectura distribuida de microservicios para un marketplace de cartas de Magic The Gathering. Permite que tiendas y usuarios individuales publiquen, compren y gestionen "singles", integrando datos externos de la API de Scryfall

2. Arquitectura del Sistema

2.1 Descripción general

El sistema fue desarrollado bajo una arquitectura de microservicios utilizando Spring Boot, permitiendo que cada módulo funcione de manera independiente.

2.2 Diagrama de Arquitectura



El sistema está compuesto por:

- API Gateway
- Eureka Server
- 10 microservicios
- Bases de datos independientes
- Comunicación mediante OpenFeign y WebClient

2.3 Roles de Usuario: Descripción de los 2 o 3 roles definidos (ej: Administrador, Cliente) y sus permisos asociados.

Administrador	Cliente/Usuario	Vendedor
Gestiona cartas (catálogo)	Compra cartas	Publica cartas (ms-listings)
Puede eliminar publicaciones	Crea pedidos	Gestiona sus publicaciones
Accede a reportes	Puede dejar reseñas (Opcional)	Recibe pagos indirectamente
Supervisa el sistema	Recibe notificaciones	Consulta sus ventas

2.4 Arquitectura de Microservicios:

Microservicio	Puerto	Función principal
Eureka-server	8761	Registro y descubrimiento de servicios.
api-gateway	8080	Punto de entrada único y enrutamiento.
ms-auth	8081	Autenticación y autorización de usuarios
ms-users	8082	Gestión de perfiles y usuarios
ms-catalog	8083	Administración del catálogo de cartas MTG
ms-listings	8084	Publicación y consulta de cartas en venta

ms-orders	8085	Gestión de órdenes de compra
ms-payments	8086	Procesamiento de pagos
ms-transactions	8087	Registro histórico de transacciones
ms-reviews	8088	Gestión de reseñas y valoraciones
ms-notifications	8089	Envío de notificaciones al usuario
ms-reports	8090	Generación de reportes y estadísticas

2.5 Diagrama Entidad-Relación (DER)

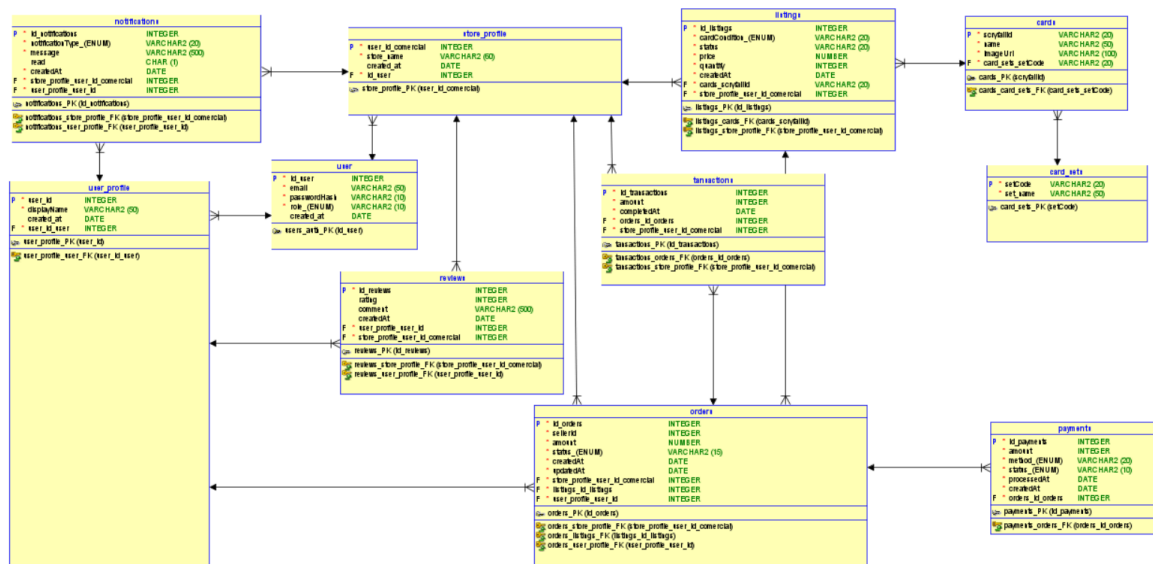


Diagrama Entidad-Relación (DER).png

3. Especificaciones Técnicas

3.1 Persistencia: Cada microservicio cuenta con su propia base de datos **MySQL**, gestionada a través de **Spring Data JPA e Hibernate**, con relaciones normalizadas y manejo de integridad referencial.

3.2 Validaciones: Implementación de **Bean Validation (JSR 380)** en DTOs y entidades para asegurar la calidad de los datos de entrada.

3.3 Comunicación: Interoperabilidad lograda mediante **OpenFeign** y **WebClient**, con manejo de timeouts y errores remotos.

Ejemplo:

1. ms-orders consulta información de usuarios mediante ms-users.
2. ms-payments valida órdenes creadas en ms-orders.
3. ms-notifications envía mensajes después de una compra exitosa.

3.4 Manejo de Errores: Uso de **@ControllerAdvice** para capturar excepciones y retornar respuestas estructuradas en formato JSON con códigos HTTP semánticos.

3.5 Logs: Trazabilidad completa mediante **SLF4J** en puntos estratégicos del flujo de negocio.

4. Tecnologías Utilizadas

4.1 Tecnologías utilizadas

- **Lenguaje de programación:** JavaScript.
- **Versión de Java utilizada:** Java 21.
- **Gestión de dependencias:** Maven.
- **Base de datos:** MySQL.
- **Framework:** Spring Boot 4.0.6.
- **Persistencia de datos:** Spring Data JPA + Hibernate.
- **Seguridad:** Spring Security + JWT para autenticación y autorización.
- **Arquitectura de microservicios:** Spring Cloud con Eureka, Gateway y OpenFeign.
- **Contenerización:** Docker y Docker Compose.
- **Documentación de API:** Swagger / OpenAPI.
- **Pruebas:** JUnit 5 y Mockito.

5. Flujo de Operación (Guía de Pruebas REST)

Para verificar el funcionamiento, se recomienda el siguiente flujo en Postman:

1. POST /auth/register → registrar usuario
2. POST /auth/login → obtener JWT
3. GET /catalog/search?name= → buscar carta
4. GET /catalog/{scryfallId} → obtener carta por ID
5. POST /listings → publicar listing
6. POST /orders → crear orden de compra
7. GET /payments/order/{id} → verificar pago
8. POST /reviews → dejar reseña
9. GET /reports/summary → ver resumen (ADMIN)

6. Swagger/OpenApi.

6.1 Configuración Swagger

La documentación de los servicios REST fue implementada utilizando Springdoc OpenApi, permitiendo generar automáticamente la documentación de los endpoints disponibles en cada microservicio.

La arquitectura cuenta con un API gateway como punto de entrada principal para administrar las rutas hacia cada microservicio.

Swagger permitirá visualizar, probar sin necesidad de herramientas externas -Postman- mostrando:

- Endpoints disponibles.
- Métodos (GET, POST, PUT/PATCH, DELETE).
- Parámetros requeridos para una operación.
- Códigos de respuesta HTTP.
- Ejemplos de request y response en formato JSON.

La interfaz gráfica de Swagger se encuentra disponible en mediante:

<http://localhost:8080/swagger-ui/index.html> → siendo el puerto de la api-gateway.

6.2 Documentación de Endpoints.

Cada microservicio cuenta con documentación propia según su responsabilidad dentro del sistema.

(imagen de algo :p)

6.4 Códigos de respuesta utilizados

Los microservicios implementados utilizan códigos de estado HTTP estándar para representar correctamente el resultado de cada operación realizada mediante la API REST.

Estos códigos permiten que los desarrolladores puedan interpretar si una solicitud fue procesada correctamente (sin el uso de Postman), si existe un error en los datos enviados o si ocurrió un problema interno del servicio.

Código	Uso
200 OK	Solicitud procesada correctamente
201 CREATED	Recurso creado exitosamente
400 BAD REQUEST	Datos enviados inválidos
401 UNAUTHORIZED	Usuario no autenticado
403 FORBIDDEN	Acceso denegado
404 NOT FOUND	Recurso no encontrado
409 CONFLICT	Conflicto de datos
500 INTERNAL SERVER ERROR	Error interno del servidor

7. Arquitectura Multimódulo.

El proyecto fue estructurado utilizando una arquitectura multimódulo gestionada por Maven.

La estructura general del proyecto es:

quandrix-parent/ (Proyecto padre)

|

|— pom.xml (Pom padre)

|

|— quandrix/

|

|— pom.xml

|

|— api-gateway/

| |— pom.xml

|

|— eureka-server/

| |— pom.xml

|

|— ms-auth/

| |— pom.xml

|

|— ms-users/

| |— pom.xml

|

|— ms-catalog/

| |— pom.xml

.

. (demás microservicios del proyecto)

.

El archivo pom.xml padre funciona como controlador principal del proyecto, definiendo los módulos existentes mediante la sección <modules> y centraliza las configuraciones tales como:

- Versión de Java utilizada.
- Dependencias compartidas.
- Plugins de compilación.
- Configuración de Spring Boot.
- Gestión de versiones Maven.

Ejemplo de configuración del POM padre:

```
<modules>
  <module>quandrix/eureka-server</module>
  <module>quandrix/api-gateway</module>
  <module>quandrix/ms-auth</module>
  <module>quandrix/ms-catalog</module>
  <module>quandrix/ms-listings</module>
  <module>quandrix/ms-notifications</module>
  <module>quandrix/ms-orders</module>
  <module>quandrix/ms-payments</module>
  <module>quandrix/ms-reports</module>
  <module>quandrix/ms-reviews</module>
  <module>quandrix/ms-transactions</module>
  <module>quandrix/ms-users</module>
</modules>
```

Cada módulo mantiene su propio pom.xml, permitiendo definir dependencias particulares según la responsabilidad del microservicio, pero heredando las configuraciones generales del proyecto padre.

7.1 Gestión de compilación mediante Maven.

Al tratarse de una arquitectura multimódulo administrada mediante Maven, la compilación del proyecto se realiza desde el directorio raíz donde se encuentra el pom.xml padre.

La primera compilación del proyecto debe realizarse utilizando:

- mvn clean install -DskipTests


```

[INFO] -----
[INFO] Reactor Summary for quandrix-parent 0.0.1-SNAPSHOT:
[INFO]
[INFO] quandrix-parent ..... SUCCESS [ 0.287 s]
[INFO] eureka-server ..... SUCCESS [ 6.534 s]
[INFO] api-gateway ..... SUCCESS [ 3.008 s]
[INFO] ms-auth ..... SUCCESS [ 4.133 s]
[INFO] ms-catalog ..... SUCCESS [ 3.700 s]
[INFO] ms-listings ..... SUCCESS [ 4.634 s]
[INFO] ms-notifications ..... SUCCESS [ 3.027 s]
[INFO] ms-orders ..... SUCCESS [ 3.540 s]
[INFO] ms-payments ..... SUCCESS [ 8.119 s]
[INFO] ms-reports ..... SUCCESS [ 11.749 s]
[INFO] ms-reviews ..... SUCCESS [ 4.261 s]
[INFO] ms-transactions ..... SUCCESS [ 3.050 s]
[INFO] ms-users ..... SUCCESS [ 3.164 s]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 59.848 s
[INFO] Finished at: 2026-06-29T15:37:35-04:00
[INFO] -----

```

Esto permite:

- Limpiar compilaciones anteriores del proyecto.
- Compilar todos los módulos definidos en el pom.xml padre.
- Descargar e instalar dependencias necesarias.
- Validar la estructura y relación entre módulos.
- Evitar la ejecución inicial de pruebas mientras se valida la integración completa del proyecto.

Una vez validada la compilación general del proyecto, se pueden ejecutar las etapas de pruebas y verificación mediante:

mvn test → pruebas unitarias de los microservicios.

y posteriormente

mvn verify → para realizar validaciones adicionales del ciclo de construcción del proyecto.

8. Pruebas Unitarias.

Las pruebas unitarias fueron implementadas utilizando JUnit y Mockito, con el objetivo de validar la lógica de negocio de cada microservicio sin depender directamente de componentes externos como bases de datos o servicios remotos.

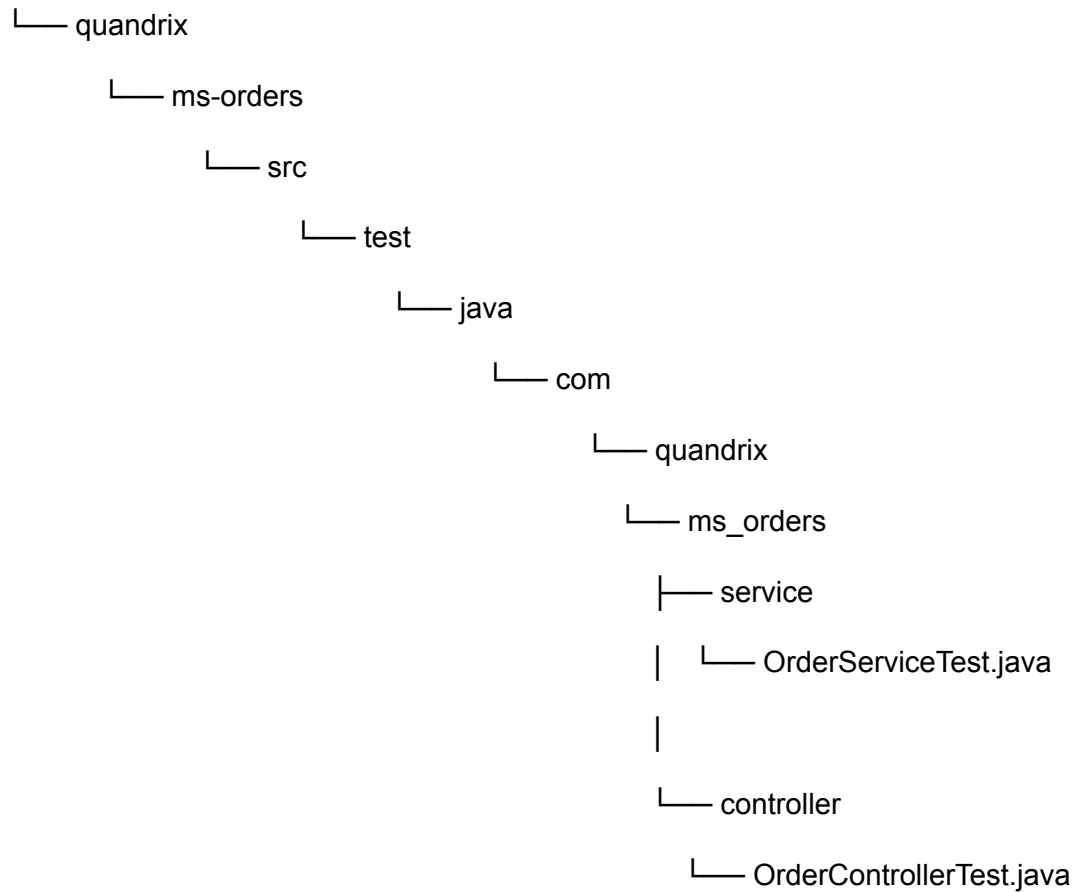
Las pruebas se encuentran organizadas dentro de:

src/test/java

Cada clase de prueba válida métodos del service y controller.

Ejemplo de estructura:

quandrix



8.1 Pruebas unitarias para Controller y Service.

Se generaron pruebas unitarias para las capas de Controller y Service de cada microservicio, alcanzando un total de: 270 pruebas unitarias.

distribuidas en 10 microservicios, con 0 fallas y 0 errores en la ejecución final de cada uno.

Microservicio	Clases cubiertas	Tests (Controller / Service)	Total
ms-auth	AuthController / AuthService / JwtService	6 / 8 / 5	19
ms-catalog	CatalogController / CatalogService / ScryfallClient	8 / 10 / 7	25
ms-listings	ListingController / ListingService	18 / 24	42
ms-notifications	NotificationController / NotificationService	13 / 16	29

ms-reports	ReportController / ReportService	9 / 12	21
ms-orders	OrderController / OrderService	13 / 18	31
ms-payments	PaymentController / PaymentService	7 / 7	14
ms-reviews	ReviewController / ReviewService	13 / 15	28
ms-transactions	TransactionController / TransactionService	13 / 10	23
ms-users	UserProfile + StoreProfile (Controller/Service)	20 / 18	38
TOTAL	10 microservicios de negocio		270

8.2 Conclusiones.

El conjunto de 270 pruebas unitarias generadas cubre, para los 10 microservicios de negocio de Quandrix, la totalidad de los métodos públicos de sus capas de Controller y Service, incluyendo:

- Casos de éxito para cada operación.
- Casos de error de validación, tanto de Bean Validation (capa Controller) como de reglas de negocio (capa Service).
- Casos de borde: listas vacías, valores nulos, duplicados, recursos inexistentes.
- Casos de tolerancia a fallos, en los flujos de orquestación más complejos (ms-orders).
- Tests de integración acotada con servidores HTTP simulados (MockWebServer), para el único cliente externo con lógica propia.

9 Automatización de ejecución del proyecto.

9.1 Scripts de inicio

Para facilitar la ejecución del sistema se desarrollaron archivos de comandos **.bat**, permitiendo iniciar el proyecto de manera automatizada según el entorno de ejecución requerido.

Se implementaron dos modalidades:

- **Ejecución sin Docker:**
Permite iniciar los servicios del proyecto utilizando las herramientas instaladas directamente en el equipo, como Java, Maven y la base de datos configurada localmente.
- **Ejecución con Docker:**
Permite levantar el entorno completo del proyecto mediante **Docker Compose**, iniciando los contenedores necesarios y sus dependencias de forma automática.

Estos scripts simplifican el proceso de puesta en marcha del sistema, reduciendo la configuración manual necesaria para ejecutar la aplicación.

10. Colaboración y Entrega.

10.1 Enlace al Repositorio de GitHub: URL del repositorio donde se visualice el historial de commits y las ramas de desarrollo.

Enlace a repositorio: <https://github.com/NovaFugaz/quandrix>

El repositorio contiene:

- **Historial** **de** **commits:**
<https://github.com/NovaFugaz/quandrix/pulls?q=is%3Apr+is%3Aclosed>

- **Ramas de desarrollo:**

Branches

New branch

Overview

Yours

Active

Stale

All

Q Search branches...

Default

Branch	Updated	Check status	Behind / Ahead	Pull request
main	19 minutes ago		Default	...

Your branches

Branch	Updated	Check status	Behind / Ahead	Pull request
dev_elba	19 hours ago		240	#16

Active branches

Branch	Updated	Check status	Behind / Ahead	Pull request
development	22 minutes ago		170	#21
dev_elba	19 hours ago		240	#16

- **Captura del despliegue (captura docker):**

```
[+] up 13/13
✓ Container quandrix-mysql Healthy
✓ Container quandrix-eureka Started
✓ Container quandrix-gateway Started
✓ Container quandrix-transactions Started
✓ Container quandrix-notifications Started
✓ Container quandrix-auth Started
✓ Container quandrix-payments Started
✓ Container quandrix-catalog Started
✓ Container quandrix-users Started
✓ Container quandrix-reviews Started
✓ Container quandrix-reports Started
✓ Container quandrix-listings Started
✓ Container quandrix-orders Started
```

```
sudo docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                                                                 NAMES
e837e226b51b   quandrix-ms-orders                  "java -jar app.jar"     36 seconds ago Up 23 seconds 0.0.0.0:8085->8085/tcp, [::]:8085->8085/tcp   quandrix-orders
bc5c9d48556b   quandrix-ms-reports                 "java -jar app.jar"     36 seconds ago Up 24 seconds 0.0.0.0:8089->8089/tcp, [::]:8089->8089/tcp   quandrix-reports
75755130b401   quandrix-ms-listings                "java -jar app.jar"     36 seconds ago Up 23 seconds 0.0.0.0:8084->8084/tcp, [::]:8084->8084/tcp   quandrix-listings
34ae68951968   quandrix-ms-reviews                 "java -jar app.jar"     36 seconds ago Up 24 seconds 0.0.0.0:8087->8087/tcp, [::]:8087->8087/tcp   quandrix-reviews
c7a38bf34384   quandrix-ms-users                   "java -jar app.jar"     36 seconds ago Up 24 seconds 0.0.0.0:8082->8082/tcp, [::]:8082->8082/tcp   quandrix-users
3259e1f7dd62   quandrix-ms-payments                "java -jar app.jar"     37 seconds ago Up 25 seconds 0.0.0.0:8086->8086/tcp, [::]:8086->8086/tcp   quandrix-payments
690e0b8e4b41   quandrix-ms-catalog                 "java -jar app.jar"     37 seconds ago Up 25 seconds 0.0.0.0:8083->8083/tcp, [::]:8083->8083/tcp   quandrix-catalog
5f5592611d80   quandrix-ms-notifications            "java -jar app.jar"     37 seconds ago Up 25 seconds 0.0.0.0:8088->8088/tcp, [::]:8088->8088/tcp   quandrix-notifications
esab8f669bba   quandrix-ms-auth                    "java -jar app.jar"     37 seconds ago Up 25 seconds 0.0.0.0:8081->8081/tcp, [::]:8081->8081/tcp   quandrix-auth
72df70295f4d   quandrix-ms-transactions             "java -jar app.jar"     37 seconds ago Up 25 seconds 0.0.0.0:8090->8090/tcp, [::]:8090->8090/tcp   quandrix-transactions
7a38d8a4bf24   quandrix-api-gateway                 "java -jar app.jar"     37 seconds ago Up 36 seconds 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   quandrix-gateway
5903c7cd5cc7   mysql:8.0                           "docker-entrypoint.s..." 37 seconds ago Up 36 seconds (healthy) 0.0.0.0:3306->3306/tcp, 33060/tcp         quandrix-mysql
5bf5c10e9b5d   quandrix-eureka-server              "java -jar app.jar"     37 seconds ago Up 36 seconds 0.0.0.0:8761->8761/tcp, [::]:8761->8761/tcp   quandrix-eureka
```

- Captura de Eureka funcionando:

DS Replicas

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
API-GATEWAY	n/a (1)	(1)	UP (1) - 7a30d8a4bf24:api-gateway:8080
MS-AUTH	n/a (1)	(1)	UP (1) - e5ab8f669bba:ms-auth:8081
MS-CATALOG	n/a (1)	(1)	UP (1) - 690e0b8e4b41:ms-catalog:8083
MS-LISTINGS	n/a (1)	(1)	UP (1) - 75755130b401:ms-listings:8084
MS-NOTIFICATIONS	n/a (1)	(1)	UP (1) - 5f5592611d80:ms-notifications:8088
MS-ORDERS	n/a (1)	(1)	UP (1) - e837e226b51b:ms-orders:8085
MS-PAYMENTS	n/a (1)	(1)	UP (1) - 3259e1f7dd62:ms-payments:8086
MS-REPORTS	n/a (1)	(1)	UP (1) - bc5c9d48556b:ms-reports:8089
MS-REVIEWS	n/a (1)	(1)	UP (1) - 34ae68951968:ms-reviews:8087
MS-TRANSACTIONS	n/a (1)	(1)	UP (1) - 72df70295f4d:ms-transactions:8090
MS-USERS	n/a (1)	(1)	UP (1) - c7a38bf34384:ms-users:8082

General Info

6.2 Gestión de Proyecto

 Gestión de proyecto